Project 01

Compilers & Interpreters

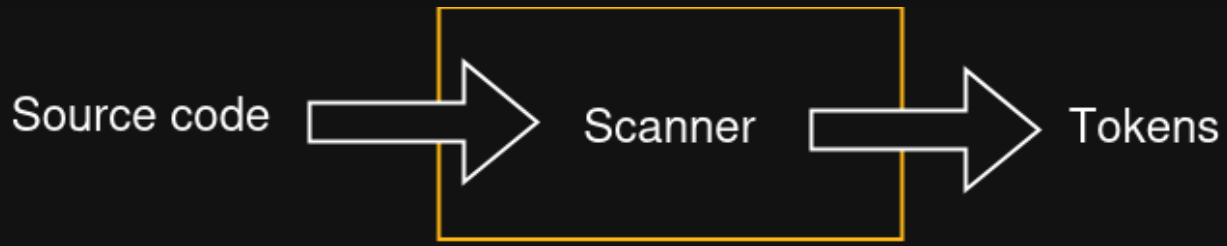
Javier Lee, Steven Loaíciga, Luan Valverde

2026, semester II

Scanning process

Scanning, or *lexical analysis*, is *conceptually* the first phase in the compilation process.

- The *scanner* receives the *source code* as input.
- It reads the entire file, character by character.
- It groups these characters into *tokens*.
- These tokens are then sent to the parser via the getToken() function.



getToken() function

This function is the main —— of the scanner.

- The entire program is a string stored in the computer's memory.
- The scanner keeps track of the current position.
- It scans through the string until it forms an entire token.
- It skips what is considered as "spaces" by the specific language...
  - Line jumps, tabs, comments, ...

Tokens

Tokens are *usually* a data structure with two fields or attributes:

- It's *token code*, which is a number that specifies if it is a variable, constant, comma, etc.
- The *lexeme*, which is usually a pointer to the particular string it refers to.

Flex

Flex is a toll for generating *lexical analyzers* (scanners).

- It lets the user define the patterns that represent tokens using *regular expressions*.
- So instead of writing the scanner from scratch, Flex generates the corresponding C code.

Flex (cont.)

- Flex receives a file with the *./l* extension.
- This file contains the patterns and corresponding actions of the language.
- Flex processes this file and generates a C file called *lex.yy.c*
- This file is the scanner, and it can be modified further by the user if they deem it necessary.

Source code

The following slides contain the source code that was processsed by the scanner.

Source code (1)

```c
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
#include "..headersscanner.h"
#include "..headersparser.h"
#include "..headerssemantic_records.h"
#include "..headerssemantic_routines.h"
#include "..headersgenerate.h"

token current_token;
static token lookahead;
static int has_lookahead = 0;

token next_token(void) {
    if (!has_lookahead) {
        lookahead = scanner();
        has_lookahead = 1;
    }
```

Source code (2)

```
    return lookahead;
}

void match(token t) {
    token received = next_token();
    char* token = extract_token(t);

    if (received == t) {
        current_token = received;
        has_lookahead = 0;
    }
    else {
        fprintf(stderr, "Syntax error at line %d: expected token %s\n",
    line, token);
        syntax_error(received);
    }
}

void syntax_error(token tok) {
    char* token = extract_token(tok);
```

Source code (3)

```
    fprintf(stderr,
            "Syntax error at line %d: unexpected token %s\n",
            line, token);
    remove("data_file");
    remove("bss_file");
    remove("text_file");
    exit(EXIT_FAILURE);
}

void id_list(void)
{
    expr_rec id;          ident(&id);
    read_id(id);
    while (next_token() == COMMA) {          match(COMMA);
                ident(&id);
        read_id(id);
    }
}

void add_op(op_rec *result){
```

Source code (4)

```
    token tok = next_token();
    if (tok == PLUSOP||tok==MINUSOP) {
        match(tok);
        *result = process_op();     } else {
        syntax_error(tok);
    }
}

void primary(expr_rec *result){
    token tok = next_token();
    switch (tok){
        case LPAREN:
            expr_rec e1, e2, e3;                match(LPAREN);
            expression(&e1);                if(next_token()==CONDITIONAL){
                match(CONDITIONAL);
                expression(&e2);
                match(CONDITIONAL);
                expression(&e3);
                match(RPAREN);
                *result = gen_conditional(e1, e2, e3);
```

Source code (5)

```
            }
            else{
            match(RPAREN);
            *result = e1;                }

            break;
        case ID:
            match(ID);
            *result = process_id();                  break;
        case INTLITERAL:
            match(INTLITERAL);
            *result = process_literal();                 break;
        default:
            syntax_error(tok);
            break;
    }
}

void expression(expr_rec *result) {
    expr_rec left_operand, right_operand;
```

Source code (6)

```
    op_rec op;

    primary(& left_operand);
    while(next_token() == PLUSOP || next_token() == MINUSOP) {
        add_op(& op);
        primary(& right_operand);
        left_operand = gen_infix(left_operand, op, right_operand);
    }
    *result = left_operand;
}

void expr_list(void){
    expr_rec expresion;
    expression(&expresion);     write_expr(expresion);
    while(next_token()==COMMA){         match(COMMA);
        expression(&expresion);
        write_expr(expresion);
    }
}
```

Source code (7)

```
void ident(expr_rec *result) {
    match(ID);
    *result = process_id();
}


void statement(void)
{
    token tok = next_token();
    switch (tok) {
        case ID:
            expr_rec target;
            expr_rec source;
            ident(&target);
            match(ASSIGNOP);
            expression(&source);
            assign(target, source);
            match(SEMICOLON);
            break;
```

Source code (8)

```
        case READ:
            match(READ);
            match(LPAREN);
            id_list();
            match(RPAREN);
            match(SEMICOLON);
            break;
        case WRITE:
            match(WRITE);
            match(LPAREN);
            expr_list();
            match(RPAREN);
            match(SEMICOLON);
            break;
        default:
            syntax_error(tok);
            break;
    }
}
```

Source code (9)

```
void statement_list(void)
{
    statement();

    while (1) {
        switch (next_token()) {
            case ID:
            case READ:
            case WRITE:
                statement();
                break;

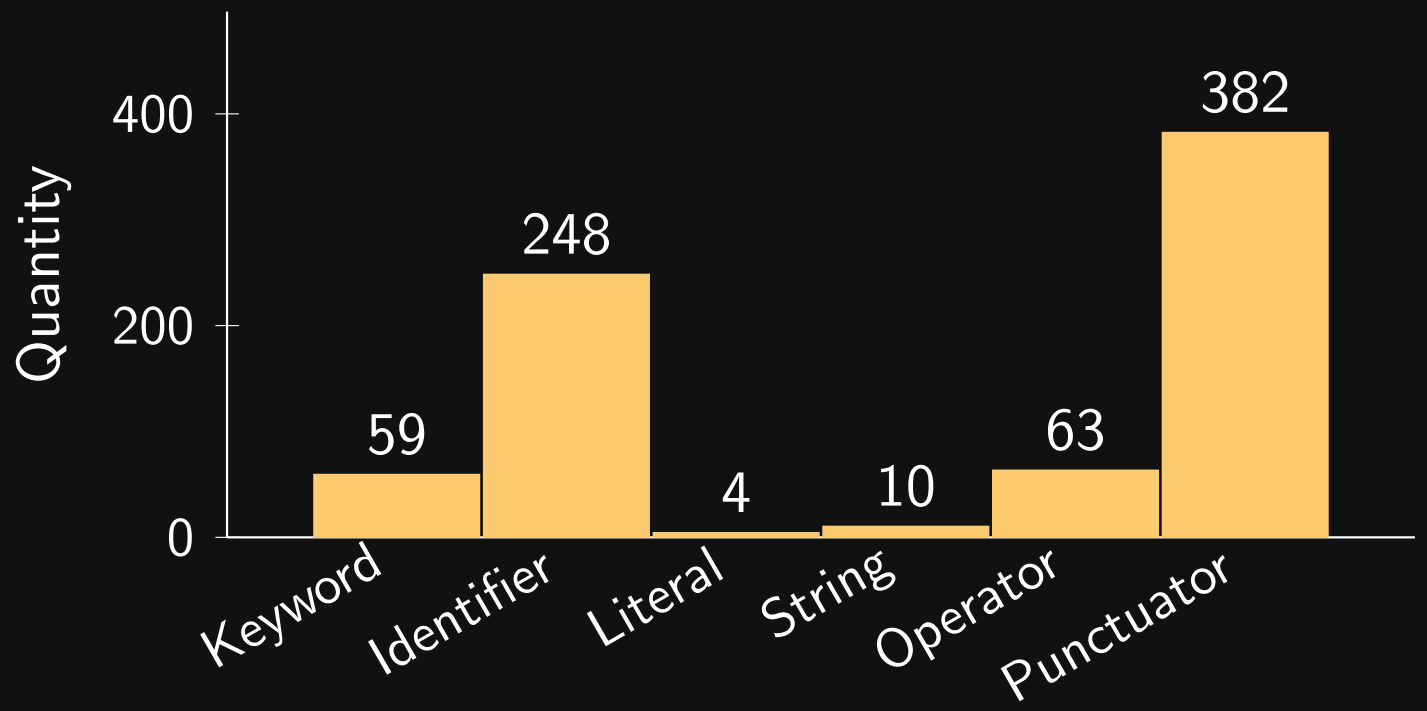
            default:
                return;
        }
    }
}

void program(void)
{
```

Source code (10)

```
    start();
    match(BEGIN);
    statement_list();
    match(END);
}
```

Statistics - Histogram



Statistics - Pie chart

That's all. Thank you for reading this presentation.